Church Machine v2.0 — Hardware Format Reference

Church Machine v2.0 — Hardware Format Reference

GoldenDetails.md — v2.0 — 2026-06-25 CONFIDENTIAL

Authoritative implementation source: `hardware/layouts.py` This document is the single canonical description of every wire-level format used by the Church Machine and the IDE. Where any other document conflicts with this one, this document takes precedence.

★ marks every field changed from v1.x

Machine Types

Two machines participate in the Church Machine platform. Every format below is annotated with which machine creates it, which machine reads it, and whether it is programmer-visible.

Tag	Machine	Role
[CM]	Church Machine	The hardware execution engine (Ti60 F225 silicon; JS simulator). Enforces capal
[IDE]	Integrated Development Environment	The web-based namespace manager. Compiles CLOOMC source to INSTR words

A third role, **Mint**, is the only entity authorised to create new GT rows at runtime. Mint is itself a CLOOMC abstraction executing on **[CM]** under IDE supervision — it is not a separate machine.

Terminology

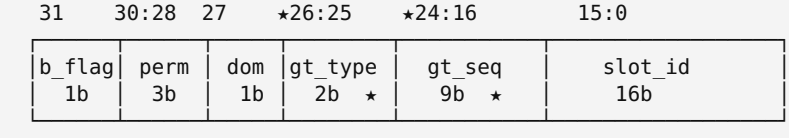
Term	Definition
NS table	Namespace table. A flat array of 4-word SLOTS in DMEM.
SLOT	One 4-word (128-bit) entry in the NS table. Addressed by `slot_id`.
C-list	Capability list. A contiguous set of GT ROWS at the start of a LUMP's c-list zone.
ROW	One GT word (32 bits) in a c-list. Addressed by c-list row index.
LUMP	Loadable Unit of Memory Protection. The fundamental deployable unit.
lump_base	The base byte address of a LUMP's first word in DMEM. Also called the ROW address of the LUMP.
Pet-Name	Human-readable capability identifier. Encoded as `cw.cc` in an Outform LUMP header.
NOP	A valid instruction that does nothing. Any instruction with `cond = NV (15)` is a NOP — the condition is never true, so the instruction is always skipped.
HALT	Does not exist. Programs do not halt; they run, fault, or loop indefinitely.
FAULT	Any invalid opcode, any permission failure, or any LUMP magic word fetched as an instruction. Triggers the three

---

Public Hardware Type 1 — GT (Golden Token)

[CM] validates · [CM/IDE] creates · Programmer-visible

A GT is a 32-bit word stored in a c-list ROW. It is the unit of authority in the Church Machine. Every memory access — load, save, call, return — requires a valid GT. GTs are unforgeable: only Mint (running on **[CM]**) can create them; only hardware can validate them.



Bits	Field	Width	Description
`[15:0]`	`slot_id`	16	NS SLOT index — 0–65535. Indexes the NS table to find the LUMP base address.

Bits	Field	Width	Description
`★[24:16]`	`gt_seq`	9	Revocation counter — 0–511. Must match the `gt_seq` stored in the NS SLOT authority word. Revoke by incrementing the SLOT's `gt_seq`; every outstanding GT for that SLOT dies instantly on next use. **Was 7 bits at [22:16].**
`★[26:25]`	`gt_type`	2	GT class (see table below). **Was at [24:23].**
`[27]`	`dom`	1	Domain selector: `0` = Turing `{X, W, R}`, `1` = Church `{E, S, L}`.
`[30:28]`	`perm`	3	Permission payload. Meaning depends on `dom` (see below).
`[31]`	`b_flag`	1	SAVE permission: `1` = SAVE through this GT row is allowed; `0` = prevented. Per GT row — different c-list rows can grant different SAVE rights for the same NS SLOT.

GT Type Encoding (`gt_type`)

`gt_type`	Name	Behaviour
`0b00`	NULL	Zero-value — no capability. The full GT word must be `0x00000000`. Any use faults immediately.
`0b01`	Inform	Concrete NS SLOT reference. `slot_id` → NS table → LUMP in DMEM.
`0b10`	Outform	Pet-Name reference. `slot_id` → NS table → Outform LUMP. **[IDE]** resolves the Pet-Name to actual LUMP c
`0b11`	Abstract	Self-describing device capability. No NS SLOT lookup, no LUMP. The GT word itself encodes the authority v

> **NULL GT = `0x00000000`** — The hardware checks the entire 32-bit word for zero. NULL is not just a `gt_type` value; it is the all-zero word. Setting any other field while leaving `gt_type = 0b00` is undefined.

Permission Bits (`dom` × `perm`)

```
dom = 0  Turing domain:   perm[2]=X  perm[1]=W  perm[0]=R
dom = 1  Church domain:   perm[2]=E  perm[1]=S  perm[0]=L
```

Perm	Domain	Meaning
R	Turing	Read data from LUMP
W	Turing	Write data into LUMP
X	Turing	Execute code in LUMP (LAMBDA / XLOADLAMBDA)
L	Church	Load a GT from a c-list ROW (LOAD / ELOADCALL)
S	Church	Store a GT into a c-list ROW (SAVE)
E	Church	Enter an abstraction (CALL / ELOADCALL)

A GT cannot simultaneously be Turing-domain and Church-domain. The `dom` bit enforces domain purity structurally — no instruction can circumvent it.

Outform and Far — Four Valid Combinations

`gt_type` and `f_flag` (in the NS SLOT) are **orthogonal axes**. All four combinations are valid:

`gt_type`	`f_flag`	Meaning
Inform	0	Local LUMP, locally resolved
Inform	1	Local LUMP, remote access node resolves
Outform	0	Pet-Name known to local **[IDE]**
Outform	1	Pet-Name known to a remote **[IDE]** node

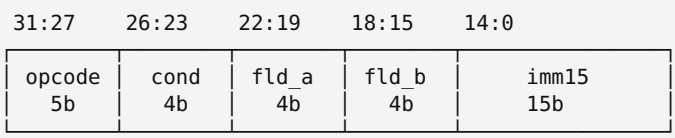
> `f_flag` **was formerly in the GT word at bit [25]**. It is now in the NS SLOT authority word at bit [31]. It is a property of the SLOT (the resolution endpoint), not of the GT row (the token). Every holder of a GT

> to a given SLOT shares the same Far/Not-Far behaviour; the hardware reads the SLOT and handles it transparently. ★

Public Hardware Type 2 — INSTR (Instruction Word)

[IDE] creates · [CM] executes · Programmer-visible

An INSTR is a 32-bit word stored in the code section of a LUMP. **[IDE]** (the CLOOMC compiler and assembler) produces INSTR words. **[CM]** fetches, decodes, and executes them.



Bits	Field	Width	Description
`[14:0]`	`imm15`	15	Immediate value; interpretation fixed per opcode.
`[18:15]`	`fld_b`	4	Second operand: CR or DR index depending on opcode.
`[22:19]`	`fld_a`	4	First operand: CR or DR index depending on opcode.
`[26:23]`	`cond`	4	Condition gate. Instruction executes only when condition is true against current flags.
`[31:27]`	`opcode`	5	Instruction selector. Values 0–9 = Church opcodes. Values 10–15 = unassigned → FAULT. Values 16–25 = Turing opcodes. Values 26–29 = unassigned → FAULT. Value 30 (0x1E) = inline data word → FAULT if fetched as instruction. Value 31 (0x1F) = LUMP magic → FAULT.

Condition Codes (`cond`)

ARM-compatible encoding. Authoritative source: `hw_types.py` `CondCode` enum and `simulator/assembler.js`.

Code	Mnemonic	Condition	Flags
0	EQ	Equal / Zero	Z=1
1	NE	Not equal / Non-zero	Z=0
2	CS	Carry set (unsigned ≥)	C=1
3	CC	Carry clear (unsigned <)	C=0
4	MI	Minus / Negative	N=1
5	PL	Plus / Non-negative	N=0
6	VS	Overflow set	V=1
7	VC	Overflow clear	V=0
8	HI	Unsigned higher	C=1 and Z=0
9	LS	Unsigned lower or same	C=0 or Z=1
10	GE	Signed greater or equal	N=V
11	LT	Signed less than	N≠V
12	GT	Signed greater than	Z=0 and N=V
13	LE	Signed less or equal	Z=1 or N≠V
14	AL	Always (unconditional)	—
15	NV	Never — **NOP**	—

> **NOP** = any INSTR with `cond = NV (15)`. The condition is never true; the instruction is always skipped with no side effects. The opcode and operand fields are irrelevant when `cond = NV`.
>
> **HALT does not exist.** Programs terminate by not returning from a top-level CALL, by entering an infinite loop, or by faulting.
>
> **LUMP magic:** `opcode = 0x1F (31)` = 11111_2. This is the top 5 bits of every LUMP header word. If **[CM]** fetches a LUMP header as an instruction, the opcode is 0x1F → undefined → **FAULT** immediately. The magic is not a sentinel value the hardware checks for; it is simply a physically impossible valid opcode that causes an automatic fault.

Opcode Table

Authoritative source: `hw_types.py` `ChurchOpcode` and `TuringOpcode` enums.

Dec	Hex	Mnemonic	Domain	fld_a	fld_b	imm15
0	0x00	LOAD	Church	CR dest	CR src (c-list)	row index
1	0x01	SAVE	Church	CR src (c-list dest)	CR src (GT)	row index
2	0x02	CALL	Church	CR (E-GT)	—	method selector
3	0x03	RETURN	Church	—	—	—
4	0x04	CHANGE	Church	CR dest	CR src	—
5	0x05	SWITCH	Church	CR (Abstract PassKey)	CR target	—
6	0x06	TPERM	Church	CR dest	CR src	preset / mask
7	0x07	LAMBDA	Church	CR (X-GT)	—	—
8	0x08	ELOADCALL	Church	CR dest	CR src (c-list)	row + selector
9	0x09	XLOADLAMBDA	Church	CR dest	CR src (c-list)	row
10–15	0x0A–0x0F	unassigned — Church extension reserved	—	—	—	→ **FAULT**
16	0x10	DREAD	Turing	DR dest	CR (R-GT)	offset
17	0x11	DWRITE	Turing	CR (W-GT)	DR src	offset
18	0x12	BFEXT	Turing	DR dest	DR src	pos + width
19	0x13	BFINS	Turing	DR dest	DR src	pos + width
20	0x14	MCMP	Turing	DR dest	DR src	—
21	0x15	IADD	Turing	DR dest	DR src	imm or —
22	0x16	ISUB	Turing	DR dest	DR src	imm or —
23	0x17	BRANCH	Turing	—	—	signed offset
24	0x18	SHL	Turing	DR dest	DR src	shift amount
25	0x19	SHR	Turing	DR dest	DR src	shift amount
26–29	0x1A–0x1D	unassigned	—	—	—	→ **FAULT**
30	0x1E	data word	—	—	—	inline data constant → **FAULT** if executed
31	0x1F	LUMP magic	—	—	—	→ **FAULT**

Hidden Implementation Detail A — NS SLOT

[IDE] creates · [CM] reads via mLoad · Not programmer-visible

The NS table is a flat array of SLOTS in DMEM. Each SLOT is 128 bits (4 × 32-bit words) at byte offset `slot_id × 16`. The programmer cannot read or write NS SLOTS directly — they are accessed only through LOAD/SAVE/CALL instructions that implicitly invoke the mLoad pipeline.

[IDE] populates the NS table in the boot image and manages it at runtime via the Loader abstraction. **[CM]** reads NS SLOTS during LOAD to validate GTs and fill CAP_REGs.

```
NS table layout — each entry = one SLOT at slot_id × 16 bytes
┌──────────────────────────────────────────────┐ ← slot_id × 16
│  Word 0   lump_base (32 bits)                │
├──────────────────────────────────────────────┤ + 4
│  Word 1   authority — WORD2_LAYOUT (32 bits) │
├──────────────────────────────────────────────┤ + 8
│  Word 2   integrity32 — CRC (32 bits)        │
├──────────────────────────────────────────────┤ + 12
│  Word 3   abstract_gt — GT_LAYOUT (32 bits)  │
└──────────────────────────────────────────────┘
```

SLOT Word 0 — lump_base

```
┌──────────────────────────────────────────────────────────────┐
│              LUMP base byte address in DMEM                  │
│                        32 bits                               │
└──────────────────────────────────────────────────────────────┘
```

The byte address of word 0 of the LUMP (the LUMP_HEADER). This is the ROW address of the LUMP in physical memory. Written by **[IDE]** at boot image build time; updated by the Loader when a lazy-load LUMP is fetched from the Tunnel.

SLOT Word 1 — authority (WORD2_LAYOUT ★)

```
 ★31       ★30      ★29:21           20:0
┌──────────┬────────┬──────────────┬──────────────────────┐
│ f_flag   │ g_bit  │   gt_seq     │       limit_offset   │
│   1b  ★  │  1b  ★ │    9b  ★     │           21b        │
└──────────┴────────┴──────────────┴──────────────────────┘
```

Bits	Field	Width	Description
`[20:0]`	`limit_offset`	21	Lump size boundary. Used by **[CM]** mLoad to enforce memory bounds.
`★[29:21]`	`gt_seq`	9	Revocation counter. **[CM]** compares `GT.gt_seq == SLOT.W1.gt_seq`; mismatch → FAULT. **[IDE]** increments to revoke all outstanding GTs for this SLOT instantly. **Was 7 bits at [27:21].**
`★[30]`	`g_bit`	1	GC mark bit. Set by **[CM]** garbage collector without invalidating integrity32 (masked before CRC). **Was at [28].**
`★[31]`	`f_flag`	1	Far indicator. `0` = local node; `1` = remote access node resolves this SLOT. Set by **[IDE]** at boot image build time. **Was in GT word at [25] — moved here.**

> `g_bit` and `f_flag` are both **masked to 0** before the integrity32 CRC is computed. Both are mutable without requiring a new CRC. `g_bit` is mutable by **[CM]**; `f_flag` is mutable by **[IDE]**.

SLOT Word 2 — integrity32

```
┌──────────────────────────────────────────────────────────────┐
│  integrity32 — 32-bit parallel check                         │
│  Formula:  ROL32(W0, 7) ^ ROL32(W1_masked, 13) ^ 0xDEADBEEF  │
```

```
┌──────────────────────────────────────────────────────────┐
│  W1_masked = W1 with g_bit[30] and f_flag[31] zeroed     │
└──────────────────────────────────────────────────────────┘
```

Written by **[IDE]**. Verified by **[CM]** on every LOAD via the ChurchNSGate pipeline. A mismatch faults with `SEAL` error — the SLOT has been tampered with.

SLOT Word 3 — abstract_gt (advisory)

```
┌──────┬──────┬──────┬──────┬──────┬──────┬──────────────┐
│b_flag│ perm │ dom  │gt_type│  gt_seq  │    slot_id    │
│  1b  │  3b  │  1b  │  2b  │   9b     │     16b       │
└──────┴──────┴──────┴──────┴──────┴──────┴──────────────┘
```

Uses full GT_LAYOUT. Advisory annotation only: `slot_id = 0`, `gt_seq = 0`, `gt_type = 0b00` (NULL). Written by **[IDE]**. **Not covered by integrity32. [CM]** only reads this word when the M-bit elevation path is active (`m_elevated`). **[IDE]** uses it as a human-readable capability label for the namespace viewer.

Hidden Implementation Detail B — LUMP_HEADER

[IDE] creates · [CM] reads at LOAD time · Not programmer-visible

The LUMP_HEADER occupies **word 0** of every LUMP at `lump_base`. It is produced by **[IDE]** (compiler / assembler / boot image generator). **[CM]** reads it during a LOAD via cLoad to obtain `cc` (c-list row count) and `n_minus_6` (size exponent) for the lump-split operation.

The programmer cannot observe the LUMP_HEADER directly. If **[CM]** ever fetches it as an instruction, `opcode = 0x1F` → **FAULT** immediately.

Common Layout (all LUMP types)

```
  31:27     26:23      22:10        9:8      7:0
┌──────────┬──────────┬──────────┬──────────┬──────┐
│  magic   │ n_minus_6│    cw    │   typ    │  cc  │
│  0x1F    │    4b    │   13b    │    2b    │  8b  │
└──────────┴──────────┴──────────┴──────────┴──────┘
```

Bits	Field	Width	Description
`[7:0]`	`cc`	8	C-list row count 0–255 (or reinterpreted by `typ`).
`[9:8]`	`typ`	2	LUMP type: `00` =Code `10` =Thread `11` =Outform.
`[22:10]`	`cw`	13	Code word count 0–8191 (or reinterpreted by `typ`).
`[26:23]`	`n_minus_6`	4	Size exponent: `lumpSize = 2^(n+6)` . Range 0–9 → 64–32768 words. Sub-64-word lumps are physically unrepresentable.
`[31:27]`	`magic`	5	Always `0x1F` . Traps instantly if fetched as an INSTR.

typ = `0b00` — Code LUMP

```
┌──────────┬──────────┬──────────┬──────────┬──────────────┐
│  0x1F    │ n_minus_6│    cw    │  0b00    │ cc = c-list row ct │
└──────────┴──────────┴──────────┴──────────┴──────────────┘
```

- `cw` : number of executable INSTR words in the code section
- `cc` : number of GT ROWs in the c-list
- **[IDE]** produces; **[CM]** reads `cc` and `n_minus_6` during LOAD lump-split

typ = `0b10` — Thread LUMP

```
┌──────────┬──────────┬──────────┬──────────┬──────────────────────────┐
│  0x1F    │ n_minus_6│    cw    │  0b10    │  heapWords (≠ cc)        │
└──────────┴──────────┴──────────┴──────────┴──────────────────────────┘
```

- `cw`: thread state word count (live CR/DR state, call stack, heap)
- `cc` **repurposed** as `heapWords`: IDE-set maximum heap allocation for this thread
- The capability zone is architecture-fixed at 12 rows; `cc` / `heapWords` is not the c-list count
- **[IDE]** sets `heapWords`; **[CM]** uses it to bound heap growth

typ = `0b11` — Outform LUMP (Pet-Name)

```
┌──────────┬──────────┬──────────┬──────────┬──────────────────────────┐
│  0x1F    │ n_minus_6│  cw = N  │  0b11    │    cc = M                │
└──────────┴──────────┴──────────┴──────────┴──────────────────────────┘
                        Pet-Name:  N . M
```

- `cw` and `cc` are **not** code/c-list counts — they jointly encode a **Pet-Name**
- **Pet-Name = `cw.cc`** — displayed by the **[IDE]** as the decimal string `N.M`
  - e.g. `cw = 4`, `cc = 7` → Pet-Name `"4.7"`
  - `cw`: major component 0–8191 (13 bits)
  - `cc`: minor component 0–255 (8 bits)
  - 2,097,152 unique Pet-Names possible (8192 × 256)
- `n_minus_6`: expected lump size after **[IDE]** resolves the Pet-Name and fetches the real LUMP
- **[IDE]** resolves `cw.cc` → real LUMP content via Tunnel; **[CM]** sees only an Outform GT referencing the SLOT and triggers the Outform protocol
- The `f_flag` in the NS SLOT (not the GT) determines whether the resolving IDE is local or Far

---

Hidden Implementation Detail C — CAP_REG (CR0–CR15)

[CM] writes at LOAD · [CM] reads on every instruction · Not programmer-visible

Each capability register (CR0–CR15) is a 96-bit internal structure filled by the LOAD instruction via the mLoad pipeline. The programmer cannot directly read the internal words — they interact with CRs only through LOAD, SAVE, CALL, RETURN, CHANGE, SWITCH, TPERM, LAMBDA, ELOADCALL, and XLOADLAMBDA.

```
┌──────────────────────────────────────────────────┐
│  Word 0   GT row  (GT_LAYOUT — 32 bits)          │  [CM] copies from c-list ROW
├──────────────────────────────────────────────────┤
│  Word 1   lump_base (32 bits)                    │  [CM] fetches from NS SLOT Word 0
├──────────────────────────────────────────────────┤
│  Word 2   authority (WORD2_LAYOUT — 32 bits)     │  [CM] fetches from NS SLOT Word 1
└──────────────────────────────────────────────────┘
```

CR Word 0 — GT row

```
┌──────┬──────┬──────┬──────┬──────────┬──────────────────────┐
│b_flag│ perm │ dom  │gt_type│  gt_seq ★ │    slot_id           │
│  1b  │  3b  │  1b  │  2b  │   9b  ★  │     16b              │
└──────┴──────┴──────┴──────┴──────────┴──────────────────────┘
```

The GT word as it existed in the c-list ROW. GT_LAYOUT — identical to Public Hardware Type 1 above.

CR Word 1 — lump_base

```
┌──────────────────────────────────────────────────────────────┐
│              LUMP base byte address in DMEM                  │
│  (copied from NS[GT.slot_id] SLOT Word 0 during LOAD)        │
└──────────────────────────────────────────────────────────────┘
```

CR Word 2 — authority

```
  ⋆31       ⋆30      ⋆29:21           20:0
┌──────────┬──────────┬──────────┬──────────────────────────────┐
│ f_flag   │ g_bit    │   gt_seq │       limit_offset           │
│   1b  ⋆  │  1b  ⋆   │    9b  ⋆ │           21b                │
└──────────┴──────────┴──────────┴──────────────────────────────┘
```

Copied from NS SLOT Word 1 during LOAD. Same WORD2_LAYOUT as the NS SLOT. **[CM]** uses `limit_offset` for bounds checking and `gt_seq` for revocation on every instruction that references this CR.

> **CR Word 3 does not exist.** NS SLOT Word 3 (abstract_gt) is advisory and is never loaded into a CAP_REG.

LOAD resolution path

```
c-list ROW (GT word)
     │
     │  GT.slot_id
     ↓
NS[GT.slot_id] SLOT
     │
     ├─ W0 → CR Word 1  (lump_base)
     ├─ W1 → CR Word 2  (authority; gt_seq verified against GT.gt_seq → FAULT if mismatch)
     └─ W2   integrity32 verified by ChurchNSGate → FAULT if CRC mismatch
GT row → CR Word 0
```

---

v2.0 Change Summary

Changes from v1.x (`hardware/layouts.py` before this release):

#	Field	Before	After	Rationale
1	GT `gt_seq`	7 bits at [22:16]	**9 bits at [24:16] ★**	2 freed bits (f_flag + spare removed); revocations 128→512
2	GT `gt_type`	2 bits at [24:23]	**2 bits at [26:25] ★**	Shifted up because gt_seq grew 2 bits
3	GT `f_flag`	1 bit at [25]	**Removed from GT ★**	Far is a SLOT property, not a GT row property
4	GT `spare`	1 bit at [26]	**Removed ★**	Absorbed by gt_seq expansion
5	GT `dom`	1 bit at [27]	**Unchanged**	Approved: Turing/Church mutual exclusion
6	NS SLOT W1 `gt_seq`	7 bits at [27:21]	**9 bits at [29:21] ★**	Must match GT gt_seq width for revocation comparison
7	NS SLOT W1 `g_bit`	1 bit at [28]	**1 bit at [30] ★**	Moved to make gt_seq contiguous; integrity32 mask updates bit-30
8	NS SLOT W1 `f_flag`	absent	**1 bit at [31] ★**	Far indicator moved here from GT
9	Outform LUMP `cw` / `cc`	described as undefined/zero	**cw.cc = Pet-Name**	IDE-defined namespace for Pet-Name resolution
10	INSTR opcode 0x1F	described as “HALT sentinel”	**LUMP magic → FAULT**	No HALT instruction exists; opcode 31 is physically impossible
11	NOP encoding	described as all-zero word	**cond = NV (15), any opcode**	NOP = any instruction with never-condition; all-zero is `LOAD cond=EQ`

Machine Ownership Matrix

Format	Created by	Read by	Programmer-visible
GT (Public Type 1)	**[CM]** Mint abstraction / **[IDE]** boot image	**[CM]** mLoad on every instruction	□ Yes — c-list ROW
INSTR (Public Type 2)	**[IDE]** CLOOMC compiler	**[CM]** fetch-decode-execute	□ Yes — code in LUMP
NS SLOT	**[IDE]** boot image + Loader	**[CM]** mLoad pipeline	□ No — hidden detail
LUMP_HEADER	**[IDE]** compiler / assembler	**[CM]** at LOAD (cLoad)	□ No — hidden detail
CAP_REG (CR0–CR15)	**[CM]** LOAD instruction	**[CM]** decode + perm check	□ No — hidden detail

End of GoldenDetails.md v2.0